1. Problem Title & Link

Title: **LeetCode 1790: Check If One String Swap Can Make Strings Equal**

Link: https://leetcode.com/problems/check-if-one-string-swap-can-make-strings-equal/

2. Problem Statement (Short Summary)

You are given two strings:

- s1
- s2

Both strings have **the same length**.

You are allowed to **swap exactly two characters in s1 at most once**.

Return **true** if it is possible to make s1 equal to s2 using **at most one swap**, otherwise return **false**.

Important idea:

Check whether the two strings differ in **exactly two positions** and whether swapping them fixes the mismatch.

3. Examples (Input → Output)

Example 1

Input

s1 = "bank"

s2 = "kanb"

Output

true

Explanation

Swap b and k in "bank":

bank → kanb

Strings become equal.

Example 2

Input

s1 = "attack"

s2 = "defend"

Output

false

Explanation

Too many mismatched characters. One swap cannot fix it.

Example 3

Input

s1 = "kelb"

s2 = "kelb"

Output

true

Explanation

Strings are already equal. No swap needed.

4. Constraints

1 ≤ s1.length, s2.length ≤ 100

s1.length == s2.length

Strings consist of lowercase English letters

Observations:

- Small input size
- Linear scan solution works perfectly

5. Core Concept (Pattern / Topic)

String Comparison / Greedy Character Matching

Idea:

- Track mismatched indices
- Ensure swap can fix the mismatch

6. Thought Process (Step-by-Step Explanation)

Step 1 — Compare both strings

Traverse both strings together.

If characters differ:

s1[i] != s2[i]

Record the mismatch.

Example:

s1 = bank

s2 = kanb

Mismatch positions:

index 0 → b vs k

index 3 → k vs b

Step 2 — Key Observation

A single swap can fix the string only if:

There are **exactly 2 mismatches**.

And they satisfy:

s1[i] == s2[j]

s1[j] == s2[i]

This means swapping fixes the mismatch.

Step 3 — Edge Case

If there are **0 mismatches**, the strings are already equal.

Return **true**.

Step 4 — Invalid Cases

Return **false** if:

- mismatches > 2
- mismatches = 1
- characters do not match after swap

7. Visual / Intuition Diagram (ASCII)

Example

s1 = bank

s2 = kanb

Comparison

index 0 → b vs k ❌

index 1 → a vs a ✔

index 2 → n vs n ✔

index 3 → k vs b ❌

Mismatches:

(b,k)

(k,b)

Swap check

b ↔ k

Strings become equal.

8. Pseudocode (Language Independent)

```
create mismatch list

for each index i:

   if s1[i] != s2[i]:
```

```
    store mismatch

if mismatch count == 0:
  return true

if mismatch count == 2:

  check swap condition
  return true if valid

otherwise return false
```

9. Code Implementation

Python

```
class Solution:
  def areAlmostEqual(self, s1, s2):

    mismatches = []

    for i in range(len(s1)):

      if s1[i] != s2[i]:
        mismatches.append(i)

    if len(mismatches) == 0:
      return True

    if len(mismatches) != 2:
      return False

    i, j = mismatches

    return s1[i] == s2[j] and s1[j] == s2[i]
```

Java

```
class Solution {
  public boolean areAlmostEqual(String s1, String s2) {
```

```
        List<Integer> mismatches = new ArrayList<>();

        for(int i = 0; i < s1.length(); i++){

            if(s1.charAt(i) != s2.charAt(i)){
                mismatches.add(i);
            }
        }

        if(mismatches.size() == 0)
            return true;

        if(mismatches.size() != 2)
            return false;

        int i = mismatches.get(0);
        int j = mismatches.get(1);

        return s1.charAt(i) == s2.charAt(j) &&
               s1.charAt(j) == s2.charAt(i);
    }
}
```

10. Time & Space Complexity

Time Complexity

O(n)

We scan the string once.

Space Complexity

O(1)

Only two mismatch indices stored.

11. Common Mistakes / Edge Cases

1. Not checking **exactly two mismatches**
2. Returning true when mismatches > 2
3. Not handling identical strings

4. Wrong swap condition

12. Detailed Dry Run

Example

s1 = bank

s2 = kanb

Index	s1	s2	Match?	Mismatch List
0	b	k	❌	[0]
1	a	a	✔	[0]
2	n	n	✔	[0]
3	k	b	❌	[0,3]

Check swap

s1[0] == s2[3]

s1[3] == s2[0]

True.

Return **true**.

13. Common Use Cases (Real-Life / Interview)

Similar logic appears in:

- Typo correction systems
- Spell check algorithms
- DNA sequence comparison
- Data validation systems

14. Common Traps (Important!)

1. Ignoring equal strings
2. Allowing more than one swap
3. Using sorting (wrong approach)
4. Incorrect mismatch indexing

15. Builds To (Related LeetCode Problems)

String comparison progression:

LC 1790 Check If One String Swap Can Make Strings Equal

LC 242 Valid Anagram

LC 1657 Determine if Two Strings Are Close

LC 567 Permutation in String

LC 438 Find All Anagrams in a String

16. Alternate Approaches + Comparison

Approach	Complexity	Notes
Sorting both strings	O(n log n)	Incorrect for this problem
Mismatch tracking	O(n)	Optimal

17. Why This Solution Works

A single swap only affects **two positions**.

Therefore the strings must differ in **exactly two indices**, and swapping those characters must match the other string.

18. Variations / Follow-Up Questions

Possible interview follow-ups:

1. Allow **two swaps instead of one**
2. Minimum swaps to make strings equal
3. Check if strings are **one edit away**
4. Allow swap in **either string**
5.